Top 100 Snowflake Interview Questions & Answers

Experienced level, including Snowflake AI / Cortex

Architecture • Performance • Storage • Loading • SQL • Pipelines • Security • AI • Scenarios

Contents

Architecture & Fundamentals 3

Warehouses & Performance 7

Storage & Table Design 12

Data Loading & Ingestion 15

SQL & Programmability 19

Streams, Tasks & Pipelines 22

Security, Governance & Sharing 25

Snowflake AI / Cortex & ML 28

Scenario-Based 35

Note: Snowflake ships features quickly, and function names and availability (especially in Cortex and AI) change often, so verify specifics against current documentation and your account's edition and region before relying on them in production.

Architecture & Fundamentals

Q1 Explain Snowflake's multi-cluster, shared-data architecture (storage, compute, cloud services).

Snowflake separates three independently scaling layers, which is why it is called a multi-cluster, shared-data architecture.

- **Storage layer:** all data lives once in cloud object storage (S3, Azure Blob, GCS) as compressed, columnar, immutable micro-partitions. Every compute cluster reads the same copy, so there is no data duplication between workloads.
- **Compute layer:** virtual warehouses are MPP clusters that execute queries. They are stateless apart from a local SSD cache, can be resized or suspended in seconds, and do not compete with one another for resources.
- **Cloud services layer:** the always-on "brain" that handles authentication, RBAC, metadata, query parsing and optimization, transaction management, security and infrastructure management.

Because storage, compute and metadata scale separately, you can run ETL, BI and data science on separate warehouses against the same data with no contention.

Q2 How does Snowflake differ from traditional warehouses and from Databricks, Redshift and BigQuery?

Versus traditional (Teradata, Oracle, on-prem) warehouses: Snowflake is fully managed SaaS, separates storage from compute, has no indexes, vacuuming or partition management to maintain, and bills per second of compute rather than for provisioned hardware.

- **Databricks:** lakehouse built on Spark and Delta Lake, strongest for heavy data engineering, streaming and ML in notebooks. Snowflake is SQL-first, with minimal tuning, and has been extending into Python (Snowpark), ML and AI. The two are converging, and both support Iceberg.
- **Redshift:** historically coupled storage and compute on clusters (RA3 and Serverless soften this). It needs more tuning (distribution and sort keys, vacuum) and Snowflake gives easier concurrency scaling and cross-cloud sharing.
- **BigQuery:** serverless, slot-based or per-TB-scanned pricing. Snowflake gives explicit control over warehouse size, isolation and cost predictability, and runs on AWS, Azure and GCP.

Snowflake's differentiators are near-zero administration, multi-cloud, Time Travel, zero-copy cloning and secure data sharing.

Q3 How do Snowflake editions differ, and which features would drive an upgrade?

Editions: Standard, Enterprise, Business Critical and Virtual Private Snowflake (VPS).

- **Standard:** core platform, 1 day maximum Time Travel, no multi-cluster warehouses.
- **Enterprise:** up to 90 days Time Travel, multi-cluster warehouses, materialized views, Search Optimization, Query Acceleration, column-level security (masking) and row access policies, periodic rekeying, object tagging and classification.
- **Business Critical:** adds Tri-Secret Secure and customer-managed keys, private connectivity (AWS PrivateLink, Azure Private Link), HIPAA/PCI/HITRUST support, and database failover/failback.
- **VPS:** a completely separate, dedicated Snowflake environment for the most sensitive organizations.

Upgrade drivers: needing more than 1 day of recovery, high BI concurrency (multi-cluster), fine-grained governance (masking, row policies) means Enterprise. Regulated data, private networking, customer-held keys or cross-region failover means Business Critical.

Q4 Explain the organization, account, region and replication hierarchy.

An **organization** is the top-level container that groups all of a company's Snowflake accounts and is managed by the ORGADMIN role (billing, account creation, usage across accounts).

- An **account** is the unit of isolation. It lives on one cloud provider in one region and has its own users, roles, databases, warehouses and edition.
- Within an account the object hierarchy is database > schema > objects (tables, views, stages, etc.).
- **Replication** copies selected objects (databases, shares, users, roles, warehouses, integrations) from a source account to target accounts in the same organization, across regions and even clouds, using replication groups and failover groups.
- Accounts in an organization can also be linked in an **organization account** for central governance, and data can be shared across them through listings.

This hierarchy supports separating environments (dev/test/prod accounts), meeting data-residency rules per region, and DR across regions.

Q5 What does the cloud services layer do, and when are cloud services credits billed?

The cloud services layer handles authentication, access control, metadata management, query compilation and optimization, transaction coordination, result-cache serving and infrastructure management. It uses compute that Snowflake manages, so you do not size it.

Billing: cloud services usage is free up to 10% of your daily virtual warehouse credit consumption. Only usage above that daily threshold is billed (the adjustment is calculated daily, in UTC).

Common causes of high cloud services cost: very large numbers of tiny queries, heavy use of `SHOW`/`INFORMATION_SCHEMA` metadata queries, frequent small DDL operations, complex compilation of huge generated SQL, and excessive single-row INSERTs. Monitor with `METERING_DAILY_HISTORY` and `QUERY_HISTORY.CREDITS_USED_CLOUD_SERVICES`.

Q6 Compare permanent, transient and temporary tables and their cost and recovery implications.

- **Permanent:** default. Time Travel up to 1 day (Standard) or 90 days (Enterprise+), plus 7 days of Fail-safe. Highest storage cost, best recoverability.
- **Transient:** persist until dropped and are visible to other sessions, but have Time Travel of 0 or 1 day and **no Fail-safe**. Cheaper storage, suited to staging, ETL intermediates and data that can be rebuilt.
- **Temporary:** exist only for the session, are visible only to it, and are dropped automatically when it ends. Time Travel 0 or 1 day, no Fail-safe. Good for scratch work.

Transient and temporary tables avoid Fail-safe storage charges, which matters for high-churn tables. The trade-off is that after retention expires, data cannot be recovered by Snowflake support. Transient is also available at schema and database level, so that everything created beneath it is transient.

Q7 Compare standard, dynamic, external, Iceberg and hybrid tables.

- **Standard (native):** data stored in Snowflake's micro-partition format, with the full feature set (Time Travel, cloning, clustering).
- **Dynamic:** defined by a SELECT query and a `TARGET_LAG`. Snowflake refreshes them automatically and incrementally where possible, so they are declarative pipelines.
- **External:** metadata over files in an external stage (Parquet, CSV, JSON, etc.). Read-only, slower, no Time Travel. Good for querying data in a lake without loading.
- **Iceberg:** open-format Apache Iceberg tables on your cloud storage. Snowflake-managed (Snowflake is the catalog, read/write) or externally managed (another catalog, read, with some write support). Great for interoperability with Spark, Trino and others.
- **Hybrid (Unistore):** row-based storage with enforced primary and foreign keys and fast single-row lookups, for OLTP-style workloads alongside analytics.

Choose by workload: analytics = standard, pipelines = dynamic, lake = Iceberg/external, transactional = hybrid.

Q8 Which clients and tools do you use (Snowsight, SnowSQL, Snowflake CLI, drivers, connectors), and when?

- **Snowsight:** the web UI for ad-hoc SQL, worksheets, notebooks, dashboards, Query Profile, monitoring and administration.
- **SnowSQL:** legacy command-line client for scripts and loading files (`PUT`/`GET`). Being replaced by Snowflake CLI.
- **Snowflake CLI (`snow`):** the modern CLI for deploying and managing Snowpark apps, Streamlit apps, Native Apps and SQL scripts, and it fits CI/CD.
- **Drivers and connectors:** Python, JDBC, ODBC, Node.js, Go, .NET and Spark connectors for applications and BI tools. Python connector with `write_pandas` for data frames, and Snowpark for DataFrame-style processing inside Snowflake.
- **Others:** SQL API (REST) for service integrations, Terraform for infrastructure, dbt for transformations.

Typical use is Snowsight for exploration, Snowflake CLI/Git in CI/CD, drivers for application access, and Snowpark where logic should run next to the data.

Q9 How does metadata-driven execution answer queries like COUNT(*) or MIN/MAX without scanning data?

Every micro-partition has metadata stored in the cloud services layer: row count, per-column min and max values, distinct-value counts, null counts and byte sizes. For queries that can be answered from these statistics alone, such as `SELECT COUNT(*) FROM t` or `MIN`/`MAX` of a column, Snowflake uses the metadata and does not start a warehouse or read any data (the Query Profile shows a "Metadata-based result" node).

Caveats: a `WHERE` filter that cannot be resolved from partition-level statistics, `DISTINCT`, or functions applied to the column will cause a real scan. The same metadata is what powers **partition pruning**, where micro-partitions whose min/max range excludes the predicate are skipped.

Q10 How do parameters work at the account, user, session and object levels?

Parameters control behavior (e.g. `TIMEZONE`, `STATEMENT_TIMEOUT_IN_SECONDS`, `DATA_RETENTION_TIME_IN_DAYS`, `QUERY_TAG`). They have defaults and can be set at different levels:

- **Account:** set by ACCOUNTADMIN for everyone (`ALTER ACCOUNT SET ...`).
- **User:** overrides the account value for that user.
- **Session:** set with `ALTER SESSION` and overrides the user and account level for that session only.
- **Object:** some parameters apply to objects (warehouse, database, schema, table, task), e.g. `DATA_RETENTION_TIME_IN_DAYS` on a table or `STATEMENT_TIMEOUT_IN_SECONDS` on a warehouse.

In general the most specific level wins, with exceptions (for timeouts, when both warehouse and session set a value, the lower one applies). Use `SHOW PARAMETERS ... IN ACCOUNT|USER|SESSION|WAREHOUSE` to inspect the effective value and the level it was set at.

Warehouses & Performance

Q11 Compare scaling up and scaling out a virtual warehouse.

- **Scale up (bigger size):** XS to S to M and so on, with credits doubling at each step. Gives more compute and memory per query, so it helps **complex, large or spilling queries** and long-running single queries.
- **Scale out (more clusters):** multi-cluster warehouses add clusters to handle **more concurrent queries**. It does not make an individual query faster, it removes queuing.

Rule of thumb: slow single query, resize up and test. Many users queuing, scale out. Because per-second billing means a bigger warehouse that finishes in half the time costs about the same, trying a larger size is often cheap. Always measure with Query Profile and `QUERY_HISTORY` (queued time, spilling) before deciding.

Q12 Explain multi-cluster warehouses: Auto-scale vs. Maximized, and the Standard vs. Economy scaling policies.

Multi-cluster warehouses (Enterprise+) are set with `MIN_CLUSTER_COUNT` and `MAX_CLUSTER_COUNT`.

- **Maximized mode:** MIN = MAX (greater than 1). All clusters start together and stay running while the warehouse is on. Good for constant, predictable high concurrency.
- **Auto-scale mode:** MIN < MAX. Snowflake starts and stops clusters according to load.

Scaling policy (auto-scale only):

- **Standard:** favors performance. Starts a new cluster as soon as queries queue, and shuts one down after 2-3 consecutive checks show the load could be handled by fewer clusters.
- **Economy:** favors cost. Starts a new cluster only if there is enough load to keep it busy for about 6 minutes, and shuts down after about 5-6 minutes of low load. Some queuing is accepted.

Use Standard for interactive BI, Economy for batch or cost-sensitive workloads.

Q13 How do you tune auto-suspend and auto-resume?

`AUTO_SUSPEND` is the idle seconds before the warehouse suspends, and `AUTO_RESUME = TRUE` starts it when a query arrives. Billing has a **60-second minimum** each time a warehouse resumes, then per second.

Tuning guidance:

- ETL/batch warehouses: low value (60 s) since they run in bursts and the cache has little reuse.
- BI/dashboard warehouses: longer (for example 5-10 minutes) so the local SSD cache stays warm, which cuts repeated scan cost. Suspending drops that cache.
- Sporadic ad-hoc: 60 s.
- Never disable auto-suspend (NULL or 0) unless there is a deliberate reason, as an idle warehouse burns credits.

Balance cache benefit against idle cost, check `WAREHOUSE_METERING_HISTORY` and the percentage of data scanned from cache, and make sure auto-resume is on so users do not get errors.

Q14 Explain the result cache, local disk (warehouse) cache and metadata cache.

- **Result cache (cloud services):** stores the exact result of a query for 24 hours (renewed each reuse, up to 31 days). If an identical query runs again and the underlying data has not changed, and the user has privileges, the result returns instantly with no warehouse needed. Disabled for queries using non-deterministic functions (like `CURRENT_TIMESTAMP`) in many cases.
- **Local disk (warehouse) cache:** micro-partitions read from remote storage are cached on the warehouse's SSD and memory. Subsequent queries touching the same data read locally. It is lost when the warehouse suspends or resizes.
- **Metadata cache:** partition statistics (counts, min/max) held in the cloud services layer, used for pruning and for answering `COUNT`, `MIN`, `MAX` without scanning.

The Query Profile shows "Percentage scanned from cache" for the local cache. Use `USE_CACHED_RESULT = FALSE` when benchmarking.

Q15 A query is slow. Walk through diagnosing it with Query Profile.

1. Open the query in Snowsight (Query History) and check the **most expensive nodes** and the percentage of time in each operator.
2. Check **time breakdown**: queuing, compilation, execution. Long queue time means a warehouse concurrency issue.
3. Look at **TableScan** nodes: partitions scanned vs total. A high ratio means poor pruning, so consider clustering or better filters.
4. Check for **spilling** to local or remote storage, which suggests a larger warehouse or a query rewrite.
5. Check **Join** nodes for row explosion (output rows far larger than inputs), usually a missing or wrong join condition or duplicate keys.
6. Look at **Sort, Aggregate and Window** nodes for heavy memory use, and **Result/Filter** nodes for late filtering.
7. Check cache hit rate and whether `SELECT *` pulls unneeded columns.

Then fix: filter earlier, select fewer columns, fix joins, cluster, resize, or use Query Acceleration or Search Optimization.

Q16 What is spilling to local and remote storage, and how do you fix it?

When an operator (sort, join, aggregation) needs more memory than the warehouse nodes have, intermediate data **spills** first to the node's local SSD ("local spilling") and, if that fills, to remote cloud storage ("remote spilling"). Remote spilling is much slower and is a strong red flag.

Fixes:

- Use a **larger warehouse** (more memory per query), or a Snowpark-optimized warehouse for memory-heavy work.
- Reduce data volume: filter early, select only needed columns, pre-aggregate.
- Rewrite the query to avoid massive sorts or `DISTINCT` on wide rows, and fix any exploding joins.
- Split a huge job into smaller batches.
- Improve pruning with clustering to read less data.

Detect it in Query Profile ("Bytes spilled to local/remote storage") or `QUERY_HISTORY` columns `BYTES_SPILLED_TO_LOCAL_STORAGE` and `BYTES_SPILLED_TO_REMOTE_STORAGE`.

Q17 What is the Query Acceleration Service, and when does it help?

The Query Acceleration Service (QAS, Enterprise+) offloads parts of eligible queries to shared **serverless compute** so a warehouse can handle outliers without being resized.

It helps with: ad-hoc analytics with unpredictable size, queries that scan very large tables with selective filters or aggregation, and warehouses where a few big queries cause delays for others.

It is enabled per warehouse with `ENABLE_QUERY_ACCELERATION = TRUE` and `QUERY_ACCELERATION_MAX_SCALE_FACTOR` (caps the serverless compute as a multiple of the warehouse size). Check eligibility with `SYSTEM$ESTIMATE_QUERY_ACCELERATION` and the `QUERY_ACCELERATION_ELIGIBLE` view.

Serverless credits are billed separately, so use it when the workload is bursty. For steady load a right-sized warehouse is usually cheaper.

Q18 What is the Search Optimization Service, and what workloads benefit?

Search Optimization Service (Enterprise+) builds and maintains a persistent **search access path** on a table so selective lookups avoid scanning many micro-partitions.

Benefits workloads such as:

- **Point lookups** with equality predicates on high-cardinality columns (ID, email, order number).
- **Substring and regex** searches (`LIKE`, `ILIKE`, `RLIKE`).
- Lookups in **VARIANT** columns, plus **geospatial** functions.
- Needle-in-a-haystack queries on very large tables where clustering is not feasible or only helps one column.

You enable it per table or per column (`ALTER TABLE t ADD SEARCH OPTIMIZATION ON EQUALITY(col)`). It costs serverless maintenance credits plus extra storage, so apply it only where queries are selective and frequent. It complements clustering, which is better for range filters on one or two columns.

Q19 When would you use Snowpark-optimized warehouses or newer-generation warehouses?

Snowpark-optimized warehouses give much more memory per node (and optionally larger local cache). Use them for memory-hungry work: ML training, large Python/Java UDF or stored procedure jobs, large in-memory DataFrame operations, and anything that spills on a standard warehouse. They are billed at a higher credit rate per size, so use them only for those jobs, on a separate warehouse.

Newer-generation (Gen2) warehouses run on upgraded hardware and software, which targets faster performance for analytics and DML (merges, deletes, updates) at a different credit rate. Use them for transformation-heavy or latency-sensitive workloads after benchmarking cost per workload, and check availability by region and cloud.

In both cases, benchmark the same job on the existing warehouse, because faster completion can offset the higher rate.

Q20 Compare resource monitors and budgets for cost control.

- **Resource monitors:** set credit quotas for the account or for specific warehouses over a time interval (daily, weekly, monthly). They trigger actions at thresholds: notify, suspend (after running queries finish) or suspend immediately (cancel queries). They only cover **virtual warehouse** credits, not serverless features.
- **Budgets:** define a monthly spending limit for the account or a custom group of objects (tagged resources, serverless features and warehouses, and AI services). They send **notifications** (including to email, queues or webhooks) when actual or forecast spend nears the limit. They do not suspend resources on their own.

Best practice is to use both: resource monitors as a hard stop for warehouse spend, and budgets for visibility and forecast alerts across all consumption types, backed by tagging and `METERING_HISTORY`/`WAREHOUSE_METERING_HISTORY` reports.

Q21 How do queuing, concurrency limits and MAX_CONCURRENCY_LEVEL affect workloads?

A warehouse can run a limited number of queries at once, determined by available resources. `MAX_CONCURRENCY_LEVEL` (default 8) is the target number of concurrent statements per cluster, and lowering it gives each query more resources, raising it favors throughput.

When a warehouse is overloaded, new queries **queue** (shown as "queued overload time"). `STATEMENT_QUEUED_TIMEOUT_IN_SECONDS` cancels statements waiting too long, and `STATEMENT_TIMEOUT_IN_SECONDS` stops runaway queries.

To reduce queuing: scale out with multi-cluster, scale up so queries finish faster, isolate heavy jobs on their own warehouse, or stagger batch jobs. Watch `QUEUED_OVERLOAD_TIME` and `QUEUED_PROVISIONING_TIME` in `QUERY_HISTORY` and `WAREHOUSE_LOAD_HISTORY`.

Q22 How do you optimize joins and spot an "exploding join"?

An exploding join returns many more rows than the inputs because of a missing join predicate (cartesian product) or duplicate keys on both sides (many-to-many). In Query Profile, the Join node shows output rows far larger than either input, and it often spills.

Optimize joins:

- Verify join keys are unique on at least one side. Deduplicate with `QUALIFY ROW_NUMBER()` first.
- Filter and aggregate before joining so fewer rows go in.
- Join on columns with matching data types (avoid implicit casts that prevent pruning).
- Select only needed columns.
- Use a larger warehouse if the join legitimately spills.
- Check for accidental `OR` conditions in joins and replace them with `UNION ALL`.

Snowflake picks the build/probe sides automatically from statistics, so fixing data and predicates beats hints.

Q23 How do you measure partition pruning efficiency?

Compare **partitions scanned** to **partitions total** in the TableScan node of Query Profile, or in `QUERY_HISTORY` (`PARTITIONS_SCANNED`, `PARTITIONS_TOTAL`). A ratio close to 1 on a selective query means poor pruning. Also check bytes scanned vs table size.

For a table, assess clustering with:

```
SELECT SYSTEM$CLUSTERING_INFORMATION('db.sch.t', '(col)');
```

Look at **average depth** (lower is better, close to 1) and the overlap histogram. High depth means many partitions overlap on the column range.

Improve pruning by filtering on columns that match how data is ordered or clustered, avoiding functions on filter columns (`WHERE DATE(ts) = ...` can block pruning), defining a clustering key on large tables, or loading data in sorted order.

Q24 Compare materialized views, dynamic tables and result caching.

- **Materialized views (Enterprise+):** precomputed results of a single-table query (limited SQL: no joins in most cases) that Snowflake keeps in sync automatically in the background. The optimizer can also rewrite queries to use them. Incur serverless maintenance cost and storage.
- **Dynamic tables:** support complex SQL including joins and aggregations, with a user-defined **target lag** and incremental refresh where possible. Best for building pipelines and transformed layers.
- **Result cache:** free, 24-hour reuse of an identical query's result. It is not a modeled object and works only if the data and query are unchanged.

Use result cache for repeated dashboard queries, materialized views for fast aggregations over a single large table, and dynamic tables for multi-step transformations needing controllable freshness.

Q25 How would you design workload isolation across ETL, BI and data science?

Create **separate warehouses per workload** so they never compete, all reading the same data:

- **ETL/ELT:** medium to large, single cluster, auto-suspend 60 s, scheduled. Consider a Gen2 warehouse for heavy DML.
- **BI/dashboards:** multi-cluster auto-scale with Standard policy, longer auto-suspend to keep cache warm, possibly with Query Acceleration for outliers.
- **Data science/ML:** separate warehouse, possibly Snowpark-optimized for training, with strict timeouts.
- **Ad-hoc/analysts:** small default warehouse with statement timeouts and resource monitors.

Add RBAC so each role can use only its warehouse, resource monitors and budgets per team, `QUERY_TAG` for chargeback, and for strict separation use separate accounts or replication. Review sizing regularly from `QUERY_HISTORY`.

Storage & Table Design

Q26 What are micro-partitions, and why does Snowflake use them?

Micro-partitions are Snowflake's automatic, contiguous storage units. Each holds roughly 50-500 MB of uncompressed data, stored **columnar and compressed**, and is **immutable**. Rows are grouped as they are loaded, with no manual partitioning needed.

Each micro-partition stores metadata (min/max per column, distinct counts, null counts). Reasons Snowflake uses them:

- **Pruning:** queries skip partitions whose ranges cannot match the filter.
- **Columnar scans:** only needed columns are read.
- **Immutability** enables Time Travel, zero-copy cloning and consistent snapshots, since updates write new partitions and keep old ones until retention ends.
- **Automatic management:** no partition keys, vacuum or index tuning.

Natural ordering of load data affects pruning, which is why clustering keys exist for large tables.

Q27 When should you define a clustering key? How do you evaluate it (SYSTEM$CLUSTERING_INFORMATION)?

Define a clustering key on **very large tables (typically multi-terabyte)** that are queried repeatedly with selective filters or range predicates on certain columns, where the data's natural load order does not match those filters, and the table is not changing too fast.

Good keys: columns commonly in `WHERE`, join or `ORDER BY` clauses, with moderate cardinality (dates, region), or expressions like `TO_DATE(ts)`. Avoid very high-cardinality (unique ID) or very low-cardinality (boolean) keys, and avoid more than 3-4 columns.

Evaluate:

```
SELECT SYSTEM$CLUSTERING_INFORMATION('t', '(order_date, region)');
```

Review `average_depth` (lower is better), `average_overlaps` and the partition depth histogram. `SYSTEM$CLUSTERING_DEPTH` gives just the depth. Compare partitions scanned before and after on representative queries, and weigh the benefit against reclustering cost.

Q28 How does Automatic Clustering work, and what does it cost?

Once a clustering key is set (`ALTER TABLE t CLUSTER BY (...)`), the **Automatic Clustering** service monitors the table and reorganizes micro-partitions in the background using serverless compute, so data stays well ordered as DML occurs. You do not schedule it, and you can suspend or resume it per table.

Cost: serverless compute credits for reclustering, plus extra storage because rewritten micro-partitions are retained for Time Travel and Fail-safe. Tables with frequent heavy updates or inserts across the key range can recluster constantly and become expensive.

Control the cost by choosing the key carefully, clustering only large tables where queries really benefit, loading data in a pre-sorted order, and monitoring `AUTOMATIC_CLUSTERING_HISTORY`. Suspend with `ALTER TABLE t SUSPEND RECLUSTER` if cost outweighs benefit.

Q29 Explain Time Travel and Fail-safe, including retention periods by edition.

Time Travel lets you query, clone or restore historical data within a retention period: `SELECT ... AT(TIMESTAMP => ...)`, `BEFORE(STATEMENT => ...)`, `CLONE ... AT`, and `UNDROP`.

- Standard edition: up to 1 day. Enterprise and above: up to 90 days for permanent objects. Transient and temporary tables: 0 or 1 day.
- Set with `DATA_RETENTION_TIME_IN_DAYS`.

Fail-safe is a non-configurable **7-day** period after Time Travel ends, for permanent tables only. Data can be recovered only by Snowflake Support, as a last-resort disaster recovery measure, and it is not user-queryable.

Both consume storage. Total recoverability for a permanent table is retention plus 7 days. Use longer retention for critical tables and short or zero for volatile, rebuildable data.

Q30 How does zero-copy cloning work, and what are its storage implications?

`CREATE TABLE/SCHEMA/DATABASE x CLONE y` creates a new object whose metadata **points to the same micro-partitions** as the source at that moment. No data is copied, so a clone is instant and initially adds no storage.

After cloning, the two objects diverge independently: modified or new micro-partitions are stored separately and billed to the object that changed them. If the source later deletes data, the cloned object keeps references, so those partitions stay in storage until nothing references them (including Time Travel and Fail-safe).

Uses: dev/test environments, safe testing of changes, backups before risky operations, point-in-time clones with `AT`/`BEFORE`. Caveats: storage can remain after dropping a source if clones still reference it, and some objects (external tables, certain stages and pipes) have clone rules.

Q31 How do you recover dropped objects or bad updates (UNDROP, Time Travel)?

- **Dropped table, schema or database:** `UNDROP TABLE t;` (also `UNDROP SCHEMA`/`DATABASE`), allowed within the retention period. If an object with the same name was recreated, rename it first.
- **Bad UPDATE/DELETE/MERGE:** find the query ID in `QUERY_HISTORY`, then recover the earlier state:

```
CREATE TABLE t_restore CLONE t BEFORE (STATEMENT => '<query_id>');
```

Then swap with `ALTER TABLE t SWAP WITH t_restore`, or insert missing rows back with `INSERT INTO t SELECT * FROM t AT(TIMESTAMP => ...) WHERE ...`.

- Do it quickly, because after retention ends only Fail-safe (Snowflake support) can help, and only for permanent tables.

Prevent repeats with longer retention on critical tables, restricted privileges, and testing destructive changes on clones.

Q32 What drives storage cost (Time Travel, Fail-safe, clones), and how do you reduce it?

Storage is billed on average compressed bytes per month, and it includes active data plus **Time Travel**, **Fail-safe** and retained data for **clones**.

Big drivers: high-churn tables (every update or delete keeps old partitions), long retention on large tables, permanent staging tables, clones that diverged, unused or forgotten tables, and stage files left behind.

Reduce it:

- Use **transient** tables (no Fail-safe) and 0-1 day retention for staging and rebuildable data.
- Lower retention on high-churn tables.
- Drop unused tables, old clones and temp objects, and purge stage files (`PURGE = TRUE`, `REMOVE`).
- Avoid frequent full-table rewrites; prefer incremental MERGE.
- Review clustering churn.

Diagnose with `TABLE_STORAGE_METRICS` (ACTIVE_BYTES, TIME_TRAVEL_BYTES, FAILSAFE_BYTES, RETAINED_FOR_CLONE_BYTES).

Q33 When do you choose transient over permanent tables?

Choose **transient** when the data can be recreated from a source and the cost of Fail-safe and long Time Travel is not justified.

Typical cases: staging and landing tables, intermediate ETL/ELT tables, dbt temporary models, high-churn tables that are fully reloaded, development and test copies, and derived marts that can be rebuilt.

Choose **permanent** for source-of-truth data, regulated or audited data, anything that cannot be reproduced, and tables where 90-day Time Travel or Fail-safe protection matters.

Trade-off: transient tables save storage (no 7-day Fail-safe) but have at most 1 day of Time Travel and cannot be recovered by support. You can create transient schemas or databases so that everything inside defaults to transient.

Q34 Compare Snowflake-managed and externally managed Iceberg tables.

Both use the open Apache Iceberg format (Parquet data files plus Iceberg metadata) stored in your own cloud storage via an external volume.

- **Snowflake-managed (Snowflake as catalog):** Snowflake reads and writes (full DML), manages metadata, compaction and snapshots. You get near-native performance and features, and other engines (Spark, Trino, Flink) can read via Snowflake's catalog SDK or Open Catalog.
- **Externally managed:** another catalog (AWS Glue, Open Catalog, REST catalog, object storage) owns the table metadata. Snowflake uses a catalog integration to read it, with refresh to pick up new snapshots (manual or auto), and write support is limited or newer.

Choose Snowflake-managed when Snowflake is the main engine and you want full features. Choose externally managed when another engine is the writer or you have a central catalog already.

Q35 What are Snowflake Open Catalog (Polaris) and its role in interoperability?

Snowflake Open Catalog is a managed service for **Apache Polaris**, an open-source REST catalog for Apache Iceberg. It provides a vendor-neutral, central catalog so multiple engines (Snowflake, Spark, Flink, Trino and others) can read and write the same Iceberg tables with consistent metadata and access control.

It supports the Iceberg REST API, role-based access controlled at the catalog level, and credential vending for storage access. Role in interoperability: avoids lock-in by keeping both data (open file format) and catalog (open standard) portable, and lets Snowflake read tables written by other engines, or let others read Snowflake-written Iceberg tables. Snowflake can integrate it as an external catalog, and you can also self-host open-source Polaris.

Data Loading & Ingestion

Q36 Compare COPY INTO, Snowpipe and Snowpipe Streaming.

- **COPY INTO <table>:** batch bulk load from a stage, run by **your warehouse**. Best for scheduled or large loads with full control (transformations, `ON_ERROR`, validation). Billed as warehouse compute.
- **Snowpipe:** continuous, serverless micro-batch loading of **files** as they arrive in a stage, triggered by cloud notifications (auto-ingest) or REST API calls. Latency is typically about a minute or a few. Billed per-second serverless compute, plus a small per-file overhead, so avoid tiny files.
- **Snowpipe Streaming:** low-latency ingestion of **rows** directly via SDK or Kafka connector, with no files or stage. Latency can be seconds or less, lower cost for high-volume streams.

Choose COPY for batch, Snowpipe for file-based near-real-time, Snowpipe Streaming for real-time event and CDC streams.

Q37 Explain internal vs. external stages (user, table and named stages).

A stage is a location holding data files for loading or unloading.

Internal stages are storage inside Snowflake:

- **User stage** (`@~`): one per user, private, not shareable.
- **Table stage** (`@%table`): tied to one table, for users with access to that table.
- **Named internal stage** (`@my_stage`): a standalone object with privileges, the most flexible choice.

External stages point to your cloud storage (S3, Azure, GCS) through a **storage integration** (preferred, avoids storing keys) or credentials, and the data stays outside Snowflake.

Use `PUT` to upload to internal stages, `LIST` to see files, `GET`/`REMOVE` to manage them. Use external stages when data already lands in cloud storage or other systems must read the files.

Q38 What file sizing and format best practices apply to loading?

- **File size:** aim for about **100-250 MB compressed** per file, so many files can load in parallel. Avoid thousands of tiny files (high overhead) and a single giant file (no parallelism). Split large files, and for Snowpipe, aggregate small ones where possible (files over a minute apart in arrival are fine).
- **Parallelism:** the number of files should be at least a multiple of the warehouse's parallel load threads, since one file is processed by one thread.
- **Format:** columnar Parquet or well-formed compressed CSV/JSON (gzip). Parquet preserves types. Specify file formats explicitly (delimiter, null handling, date formats).
- **Organization:** partition paths by date, and use patterns or path-based loading to limit scanned files.
- Use the right warehouse size (bigger does not speed up few files), and clean up with `PURGE`.

Q39 How do you handle load errors (ON_ERROR, VALIDATE)?

`ON_ERROR` in `COPY INTO` controls behavior on bad rows:

- `ABORT_STATEMENT` (default for bulk): stop at first error.
- `CONTINUE`: skip bad rows and load the rest.
- `SKIP_FILE` / `SKIP_FILE_n` / `SKIP_FILE_n%`: skip a file with errors above a threshold.

Validate before loading: `COPY ... VALIDATION_MODE = RETURN_ERRORS` (or `RETURN_n_ROWS`) checks files without loading.

Investigate after loading: `SELECT * FROM TABLE(VALIDATE(t, JOB_ID => '_last'));` returns the rows rejected in a past COPY job.

Also use `COPY_HISTORY` for file-level status, store rejected records in an error table, set `ERROR_ON_COLUMN_COUNT_MISMATCH`, and alert on failures. Snowpipe errors are visible in `PIPE_STATUS` and `COPY_HISTORY`.

Q40 How does Snowflake track load metadata and prevent duplicate loads?

Snowflake records, per table, which staged files were loaded (name, size, ETag or checksum, load time) in **load metadata**. This lets it skip files already loaded successfully.

- For `COPY INTO`, metadata is kept for **64 days**. Files already loaded are skipped unless `FORCE = TRUE`. Files older than 64 days with unknown status may reload unless `LOAD_UNCERTAIN_FILES` or the file name changes.
- For Snowpipe, load history is kept for **14 days**.

A modified file with the same name is treated based on checksum/ETag rules, so renaming or re-uploading changes behavior. For guaranteed idempotency use unique file names, `PURGE`, or archive loaded files, and for the full dedupe use `MERGE` with business keys. Review `COPY_HISTORY` and `LOAD_HISTORY`.

Q41 How does Snowpipe auto-ingest work with cloud event notifications?

1. Files land in an external stage location (S3, Azure Blob, GCS).
2. The cloud provider emits an event notification (S3 event to SQS, Azure Event Grid to a storage queue, GCS Pub/Sub).
3. The pipe, created with `AUTO_INGEST = TRUE`, listens to that queue (Snowflake manages the queue for S3, you configure the notification channel to it).
4. Snowflake queues the new file and **serverless compute** runs the pipe's `COPY INTO`, loading the file into the target table.
5. Load status is recorded and viewable in `COPY_HISTORY` and `SYSTEM$PIPE_STATUS`.

Setup: create the stage, target table and pipe, then configure bucket notifications to the pipe's `notification_channel`. Use `ALTER PIPE ... REFRESH` to backfill pre-existing files. For internal stages use the REST API instead.

Q42 How does the Kafka connector work with Snowpipe Streaming?

The Snowflake Connector for Kafka runs inside Kafka Connect and consumes topic records, writing them to Snowflake tables. It has two ingestion modes:

- **Snowpipe (file-based):** buffers records into files in an internal stage and uses Snowpipe to load them. Higher latency, more per-file cost.
- **Snowpipe Streaming:** the connector opens streaming channels and inserts rows directly into the table with no staging files, giving lower latency (seconds) and lower cost, with exactly-once semantics per channel using offset tokens.

Default table layout is two VARIANT columns (`RECORD_METADATA` and `RECORD_CONTENT`), and schema detection/evolution can map fields to columns. Tune with buffer size, flush time and number of tasks, and

monitor lag and error handling (dead-letter queue).

Q43 How do you unload data (COPY INTO <location>)?

`COPY INTO @stage/path/ FROM table_or_query` writes query results to files in an internal or external stage (or directly to a cloud location through a storage integration).

Key options:

- `FILE_FORMAT` (CSV, JSON, Parquet), `COMPRESSION`, `HEADER = TRUE`.
- `MAX_FILE_SIZE` to control file sizes (default about 16 MB, can be increased up to 5 GB).
- `SINGLE = TRUE` for a single file.
- `PARTITION BY (expr)` to write into folder structures (for example by date).
- `OVERWRITE`, `INCLUDE_QUERY_ID`.

Example: `COPY INTO @out/sales/ FROM (SELECT * FROM sales) FILE_FORMAT=(TYPE=PARQUET) HEADER=TRUE;`

Then `GET` to download from internal stages. Unloading uses your warehouse. For data lakes, consider writing Iceberg tables instead.

Q44 Compare external tables and Iceberg tables for data lake access.

External tables are read-only metadata layers over files in external stages, usually presented as a VARIANT column plus virtual columns. Refreshing metadata is needed (auto-refresh via notifications), queries are slower since files are scanned with limited pruning (partitioning helps), and there is no Time Travel or DML.

Iceberg tables use a proper table format with schema, snapshots, manifests and statistics, which gives **better performance, partition and file pruning, schema evolution, and for Snowflake-managed tables full read/write with DML** and transactional guarantees. They interoperate with other engines.

Recommendation: use Iceberg tables for new lake or lakehouse designs. Keep external tables for simple, occasional querying of raw files or formats Iceberg does not cover.

Q45 What is Snowflake Openflow, and how does it help with ingestion from many sources?

Openflow is Snowflake's managed, **multi-modal data integration service built on Apache NiFi**. It provides a catalog of prebuilt connectors and processors to ingest structured and unstructured data (databases with CDC, SaaS apps such as Salesforce, Google Drive, Slack, SharePoint, streams like Kafka, files and more) into Snowflake, with visual flow design.

Highlights: runs in your cloud (BYOC) or on Snowpark Container Services, supports batch and streaming, handles schema drift, uses Snowflake's security and governance, and can extract and chunk documents for downstream AI/RAG. It reduces reliance on separate ETL tools and custom scripts, and gives one platform for many sources with monitoring and extensible custom processors. Check current connector availability and pricing for your region.

Q46 How do INFER_SCHEMA and schema evolution work?

`INFER_SCHEMA` scans staged files (Parquet, Avro, ORC, CSV, JSON) and returns column names, types and nullability:

```
SELECT * FROM TABLE(INFER_SCHEMA(LOCATION=>'@stg/data/', FILE_FORMAT=>'pq'));
```

Combine with `CREATE TABLE ... USING TEMPLATE (SELECT ARRAY_AGG(OBJECT_CONSTRUCT(*)) FROM TABLE(INFER_SCHEMA(...)))` to create a table automatically.

Schema evolution: set `ENABLE_SCHEMA_EVOLUTION = TRUE` on the table (and the role needs `EVOLVE SCHEMA` privilege) and load with `MATCH_BY_COLUMN_NAME`. Snowflake can then add new columns and drop NOT NULL on columns missing in a file. It works for COPY, Snowpipe, Snowpipe Streaming and the Kafka connector. Limitations: it adds columns but does not change types or drop columns, so still control it with governance and monitoring.

Q47 How would you implement CDC ingestion into Snowflake?

Typical pattern:

1. **Capture changes** from the source with a CDC tool (Openflow CDC connectors, Fivetran, Debezium via Kafka, AWS DMS, Oracle GoldenGate), producing events with operation type (I/U/D), key, timestamp and sequence.
2. **Land** them in a raw/bronze table through Snowpipe, Snowpipe Streaming or the Kafka connector (append-only).
3. **Apply** them to a current-state table using a **stream** on the raw table plus a **task** (or a dynamic table) that runs `MERGE`, using the latest change per key (`QUALIFY ROW_NUMBER() OVER (PARTITION BY key ORDER BY seq DESC) = 1`), handling deletes explicitly.
4. Keep history (SCD Type 2) if needed, and handle late or out-of-order events by sequence number.
5. Monitor lag, add alerts, and reconcile counts with the source periodically.

SQL & Programmability

Q48 Explain VARIANT, FLATTEN and LATERAL for semi-structured data.

`VARIANT` stores semi-structured data (JSON, Avro, Parquet, XML) up to 16 MB per value. Snowflake stores it in an optimized columnar form, so common paths are pruned and scanned efficiently.

Access with path notation and casts: `v:customer.name::string`, `v:items[0].sku`.

`FLATTEN` is a table function that expands an array or object into rows. `LATERAL` lets it reference columns of the preceding table in the same `FROM` clause:

```
SELECT o.id, i.value:sku::string AS sku
FROM orders o, LATERAL FLATTEN(INPUT => o.v:items) i;
```

Useful options: `OUTER => TRUE` (keep rows with empty arrays), `RECURSIVE => TRUE`, and `PATH`. Output columns include `VALUE`, `KEY`, `INDEX` and `PATH`.

Q49 How do you keep JSON queries performant?

- **Extract hot fields** into typed columns (during load or in a view/dynamic table). Snowflake auto-columnarizes up to a limit, but explicit columns give the best pruning and join performance.
- **Cast paths** (`::number`, `::string`) and avoid repeated parsing in expressions.
- Keep **VARIANT values small** and consistent, avoid very large nested documents, and use `FLATTEN` only on what is needed.
- Filter early on extracted columns, and avoid `SELECT *` on VARIANT.
- Use **Search Optimization** for point lookups inside VARIANT.
- Cluster on an extracted expression when filtering is common.
- Prefer Parquet or typed ingestion where the schema is stable, and use `INFER_SCHEMA`/schema evolution to promote JSON to columns.

Check Query Profile for bytes scanned and spilling.

Q50 Explain window functions and QUALIFY.

Window functions compute values across a set of related rows without collapsing them: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG/LEAD`, `SUM() OVER`, `FIRST_VALUE`, with `PARTITION BY`, `ORDER BY` and frame clauses (`ROWS BETWEEN ...`).

`QUALIFY` filters on window function results directly, like `HAVING` for aggregates, so you avoid a subquery:

```
SELECT * FROM events
QUALIFY ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY ts DESC) = 1;
```

It is evaluated after window functions and before `DISTINCT`/`LIMIT`. Common uses: deduplication, latest record per key, top-N per group, running totals and period-over-period comparisons.

Q51 How do you implement upserts and SCD Type 2 with MERGE?

Upsert:

```
MERGE INTO tgt t USING src s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.val = s.val
WHEN NOT MATCHED THEN INSERT (id, val) VALUES (s.id, s.val);
```

Ensure the source has one row per key (deduplicate first), otherwise nondeterministic merge errors can occur (`ERROR_ON_NONDETERMINISTIC_MERGE`).

SCD Type 2: keep `effective_from`, `effective_to` and `is_current` columns. Stage changes so that each changed key produces two rows: one to close the existing record (matched on key and `is_current` and a changed hash) and one to insert the new version (matched as NOT MATCHED). A common trick is to union the source with itself using a null merge key for the insert row. Then `WHEN MATCHED AND hash differs THEN UPDATE SET is_current=FALSE, effective_to=...` and `WHEN NOT MATCHED THEN INSERT`. Use a hash of tracked columns to detect change, and run it incrementally from a stream.

Q52 Compare stored procedures in SQL (Snowflake Scripting), JavaScript, Python and Java.

- **Snowflake Scripting (SQL):** procedural SQL with variables, loops, branching, cursors, exceptions. Best for SQL-centric orchestration and DDL/DML logic, most natural and efficient.
- **JavaScript:** older option with a JS API for executing SQL, good for dynamic SQL and string handling, but verbose.
- **Python (Snowpark):** DataFrame API, access to packages via Anaconda channel, ideal for data engineering and ML logic. Runs in the warehouse sandbox.
- **Java/Scala (Snowpark):** typed code for teams standardized on JVM.

All can run with caller's or owner's rights (`EXECUTE AS CALLER | OWNER`). Python/Java need packages and handler definitions, with slower start-up than SQL. Choose SQL scripting by default, Python when you need libraries or complex logic, Java for existing JVM code.

Q53 Compare UDFs, UDTFs and external functions.

- **UDF (scalar):** takes input columns and returns one value per row. Written in SQL, JavaScript, Python, Java or Scala, and runs inside Snowflake.
- **UDTF (table function):** returns zero or more rows per input row (e.g. parsing, tokenizing). Used in `FROM` with `TABLE(fn(...))`, and can use partition-based processing (`END_PARTITION`) for batch-style logic. Vectorized UDFs/UDTFs in Python process batches for speed.
- **External function:** calls a remote service (e.g. AWS API Gateway/Lambda, Azure Function) through an API integration. Use when logic or a model lives outside Snowflake. Higher latency, rate and size limits, and data leaves Snowflake, so governance matters.

Prefer native UDFs for performance and security, and external functions or External Access integrations only when necessary.

Q54 Which constraints are enforced in Snowflake, and which are informational only?

- **Enforced:** `NOT NULL` for standard tables.
- **Informational only (not enforced):** `PRIMARY KEY`, `FOREIGN KEY` and `UNIQUE` on standard tables. They can be declared and are used by BI tools and optimizers (with `RELY`, Snowflake may use them for join elimination), but duplicates are not prevented.
- **Enforced on hybrid tables:** primary keys, foreign keys and unique constraints are enforced.
- `CHECK` constraints are not supported on standard tables.

So data quality must be handled in pipelines: dedupe with `QUALIFY`, validate in `MERGE`, use data metric functions, and run tests (dbt) to check uniqueness and referential integrity.

Q55 How do sequences and identity/autoincrement columns behave?

A `SEQUENCE` generates unique numbers via `NEXTVAL`, and `IDENTITY`/`AUTOINCREMENT (start, step)` columns use a similar mechanism with a default value.

Key behaviors: values are **unique but not guaranteed to be gap-free or strictly ordered**, particularly with concurrent loads or multiple warehouse nodes, and failed or rolled-back statements can consume numbers. Sequences are shared objects and can be used across tables. Using `ORDER` on an identity column or sequence (`ORDER`/`NOORDER`) can guarantee ascending order, but with a performance cost.

Do not rely on them for gapless numbering or business meaning. For surrogate keys in distributed loading, they are fine, and alternatives are `UUID_STRING()` or hash keys (`MD5`/`SHA2` of business keys).

Streams, Tasks & Pipelines

Q56 Explain standard, append-only and insert-only streams, and how stream offsets work.

A **stream** records change data capture (CDC) metadata on a table or view: it does not store data, it tracks an **offset** (a point in the table's version history) and returns the changes since that offset, with columns `METADATA$ACTION`, `METADATA$ISUPDATE` and `METADATA$ROW_ID`.

- **Standard:** tracks inserts, updates and deletes (an update appears as a delete plus insert row pair).
- **Append-only:** tracks inserts only, ignoring updates/deletes. Cheaper for ELT on insert-heavy sources.
- **Insert-only:** for external tables, tracks newly added files/rows.

Offset: the stream returns changes up to the current moment, and the offset **advances only when the stream is consumed in a committed DML** statement (INSERT/MERGE...SELECT from the stream). Selecting does not move it. Multiple consumers need separate streams. If the offset falls outside the table's retention (extendable by `MAX_DATA_EXTENSION_TIME_IN_DAYS`), the stream becomes **stale** and must be recreated.

Q57 Explain tasks and task graphs, including serverless tasks.

A **task** runs a SQL statement, stored procedure or script on a schedule (`SCHEDULE = '5 MINUTE'` or CRON) or after another task. A **task graph (DAG)** links tasks with `AFTER`, with a root task holding the schedule, and supports parallel branches, conditional execution (`WHEN SYSTEM$STREAM_HAS_DATA('s')`), retries, and **finalizer** tasks that run after the graph completes for cleanup/notification.

- **User-managed tasks** use a named warehouse you size.
- **Serverless tasks** omit the warehouse and Snowflake auto-sizes compute (with `USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE` as a starting point), billed per use. Good for small, variable jobs.

Operate with `EXECUTE TASK`, `TASK_HISTORY`, `SUSPEND/RESUME` (resume child tasks first, root last), and error notifications. Tasks combine with streams for incremental ELT.

Q58 What are dynamic tables? Explain target lag and incremental vs. full refresh.

A dynamic table is a table defined by a query that Snowflake **keeps up to date automatically**, so you declare the result you want rather than writing streams, tasks and merge logic.

- **Target lag:** the maximum acceptable staleness (for example `'5 minutes'`, or `DOWNSTREAM` to refresh only when downstream tables need it). Snowflake schedules refreshes to meet it.
- **Incremental refresh:** processes only changed source data (supported for many query shapes: filters, joins, aggregations, window functions), cheaper and faster.
- **Full refresh:** recomputes everything, used when the query is not incrementalizable, or when forced. `REFRESH_MODE` can be `AUTO`, `INCREMENTAL` or `FULL`.

Chains of dynamic tables form a pipeline whose lag is coordinated. Shorter lag means higher cost, so choose lag per business need. Monitor with `DYNAMIC_TABLE_REFRESH_HISTORY` and the refresh mode shown in `SHOW DYNAMIC TABLES`.

Q59 When do you choose streams plus tasks over dynamic tables?

Choose **dynamic tables** when the transformation is expressible as a single declarative query, you want simple, maintainable pipelines, and a lag in seconds/minutes is acceptable. They remove orchestration code.

Choose **streams plus tasks** when you need **procedural control**: custom MERGE logic or SCD2, multi-statement transactions, calls to stored procedures, UDFs or external functions with side effects, writing to multiple targets from one change set, conditional logic, notifications, precise scheduling or retries, or when the query is not supported for incremental refresh.

Also consider cost predictability: tasks run exactly when you schedule, while dynamic table refresh follows lag. Many teams mix them, using dynamic tables for standard transformations and tasks for exceptions.

Q60 How do alerts, notifications and event tables support observability?

- **Alerts:** schedule a condition (SQL query) and an action when it returns rows, for example email when a pipeline lags or credits spike (`CREATE ALERT ... IF (EXISTS (...)) THEN CALL ...`).
- **Notification integrations:** send messages to email, SNS, Pub/Sub, Event Grid, Slack or webhooks through `SYSTEM$SEND_EMAIL` and notification integrations, including task-error notifications.
- **Event tables:** a special table that collects **logs, traces and metrics** emitted by UDFs, procedures, Streamlit apps, Native Apps and SPCS services (via `logging`, `snowflake.telemetry`, OpenTelemetry). Set `LOG_LEVEL`/`TRACE_LEVEL` and query it with SQL.

Together with `TASK_HISTORY`, `COPY_HISTORY`, data metric functions and `ACCOUNT_USAGE`, they let you monitor freshness, failures, cost and performance.

Q61 How would you design a medallion architecture in Snowflake?

Organize data in layers, as separate schemas or databases:

- **Bronze (raw):** landed as-is from sources using Snowpipe/Streaming/Openflow, append-only, often VARIANT or schema-on-read, with load metadata (file name, load time). Transient where reloadable.
- **Silver (cleansed/conformed):** deduplicated, typed, validated, standardized (CDC applied, SCD2), joined to reference data. Built with dynamic tables or streams and tasks, or dbt.
- **Gold (business-ready):** marts, aggregates and semantic views for BI, ML features and AI. Governed with masking, row policies and tags, and optimized (clustering, materialized views).

Supporting design: RBAC per layer (raw access restricted), separate warehouses for ingest vs transform vs BI, data quality checks between layers (data metric functions, dbt tests), lineage via Access History, environments with clones, and CI/CD for code.

Q62 How do you handle CI/CD (dbt, Git integration, Snowflake CLI, Terraform, schemachange)?

Split **infrastructure** from **code/objects**:

- **Terraform** (Snowflake provider) manages account-level infrastructure: databases, warehouses, roles, grants, integrations, network policies.
- **Schema/object deployment:** schemachange (versioned migration scripts), Snowflake CLI (`snow sql`, `snow deploy` for Snowpark/Streamlit/Native Apps), or declarative `CREATE OR ALTER` objects.
- **dbt** (Core/Cloud, or dbt projects on Snowflake) for transformations, tests and docs.

- **Git integration:** Snowflake repositories link to GitHub/GitLab/Azure DevOps so you can run scripts directly from a Git stage (`EXECUTE IMMEDIATE FROM @repo/branches/main/...`).

Pipeline: pull request, lint/test on a **zero-copy clone** of prod or a dev database, merge, deploy to test then prod with approvals, store secrets in the CI vault, and use key-pair or OAuth service users.

Security, Governance & Sharing

Q63 Explain RBAC: role hierarchy, system roles, database roles and future grants.

Privileges are granted to **roles**, and roles are granted to users or to other roles (forming a hierarchy where parent roles inherit child privileges).

System roles: ORGADMIN (organization), ACCOUNTADMIN (top, combines SECURITYADMIN and SYSADMIN, use sparingly), SECURITYADMIN (manage grants and roles), USERADMIN (users and roles), SYSADMIN (create and own objects such as warehouses and databases), PUBLIC.

Database roles are scoped to a database, so they travel with it through cloning, replication and sharing, and are granted to account roles.

Future grants (`GRANT SELECT ON FUTURE TABLES IN SCHEMA s TO ROLE r`) apply automatically to objects created later, and `ALL` grants cover existing ones.

Best practice: functional roles (access roles hold privileges, functional roles inherit them), custom roles rolled up under SYSADMIN, least privilege, managed access schemas, and no daily work as ACCOUNTADMIN.

Q64 What are secondary roles, and when are they useful?

A session has one **primary role**, and with `USE SECONDARY ROLES ALL` (or a specific list) the user also gets the **combined privileges of all their other granted roles** (for most object access, but object creation uses the primary role).

Useful when users hold multiple roles for different data domains and need to query across them without switching roles constantly, such as an analyst joining marketing and finance tables. It simplifies BI tools that cannot switch roles. Set `DEFAULT_SECONDARY_ROLES = ('ALL')` on the user.

Cautions: policies (masking, row access) evaluate with the active roles, so secondary roles can widen access, and ownership of created objects follows the primary role. Use deliberately within least-privilege design.

Q65 How do network policies, MFA, key-pair auth, OAuth, SSO and SCIM fit together?

They are layers of access control:

- **Network policies** and rules limit which IP ranges (or private endpoints) may connect, at account, user or integration level.
- **Authentication policies** define allowed methods, MFA requirements, client types and session rules.
- **MFA** (Duo-based, enforced for password logins on human users) adds a second factor.
- **SSO** via SAML 2.0 with an IdP (Okta, Entra ID) centralizes human login.
- **Key-pair authentication** (RSA, rotation supported) and **OAuth** (Snowflake or external) suit services and applications. Newer options include programmatic access tokens and workload identity.
- **SCIM** automatically provisions and deprovisions users and roles from the IdP.

Design: humans through SSO plus MFA, service users (type SERVICE) with key-pair or OAuth, network restrictions everywhere, and SCIM to keep identity lifecycle in sync.

Q66 How do dynamic data masking, row access policies and tag-based masking work?

- **Dynamic data masking:** a masking policy is a schema-level object with a function using `CURRENT_ROLE()` (or `IS_ROLE_IN_SESSION`) to return the real value, a partial value or a hash at query time. It is attached to a column, and the stored data is unchanged.
- **Row access policies:** a policy returns TRUE/FALSE per row, often by looking up a mapping table of role/user to allowed regions, so each user sees only their rows.
- **Tag-based masking:** attach a masking policy to a **tag**, then tag columns (or tables/schemas, with inheritance). Any column carrying the tag, matching the data type, is masked automatically, which scales governance.

All need Enterprise+. Other tools: projection and aggregation policies. Policies apply to all access paths (views, shares), so test with different roles.

Q67 How do object tagging and automatic sensitive-data classification work?

Object tags are key-value metadata attached to objects (columns, tables, warehouses, users...) and **inherit** down the hierarchy (database to schema to table to column). They support governance, cost attribution (tag warehouses by team) and policy attachment.

Classification analyzes column names and sampled values to identify personal data: `SYSTEM$CLASSIFY` (or a classification profile for automatic, scheduled classification) returns semantic categories (NAME, EMAIL, PHONE) and privacy categories (IDENTIFIER, QUASI_IDENTIFIER, SENSITIVE), and can **auto-apply system tags** (`SNOWFLAKE.CORE.SEMANTIC_CATEGORY`, `PRIVACY_CATEGORY`).

Together with tag-based masking this yields a pipeline: new data lands, gets classified and tagged, then masking policies attach automatically. Always have stewards review results, since classification is probabilistic.

Q68 Explain Snowflake's hierarchical key encryption model, Tri-Secret Secure and customer-managed keys.

All data is encrypted at rest (AES-256) and in transit (TLS) by default. Keys form a **hierarchy**: root key (in an HSM) wraps **account master keys**, which wrap **table master keys**, which wrap per-file **data keys**. Keys are rotated automatically (every 30 days) and, on Enterprise+, **periodic rekeying** re-encrypts data annually.

Tri-Secret Secure (Business Critical+): the account master key becomes a **composite** of a Snowflake-maintained key and a **customer-managed key** held in your cloud KMS (AWS KMS, Azure Key Vault, Google Cloud KMS). If you revoke your key, Snowflake can no longer decrypt the data (crypto-shredding / kill switch).

Trade-offs: you take on key availability responsibility, since losing or disabling the key makes data inaccessible.

Q69 Explain secure data sharing, reader accounts, listings and the Marketplace.

Secure data sharing lets a provider grant read-only access to selected objects (tables, secure views, UDFs, semantic views) to consumer accounts **without copying data**. Consumers create a database from the share, and see live data, and compute is paid by the consumer. Sharing is metadata-based, and works within the same region/cloud, with cross-region/cloud sharing handled through replication (auto-fulfillment for listings).

- **Reader accounts:** provider-managed accounts for consumers without Snowflake, with the provider paying for their compute.
- **Listings:** the packaging for sharing, either **private** (to specific accounts) or **public on the Snowflake Marketplace**, with descriptions, sample queries, free/paid/trial access and usage analytics.
- **Marketplace:** a catalog of third-party datasets, apps and AI products you can obtain instantly.

Q70 Compare secure views and standard views.

Both are named SELECT definitions, but **secure views** add protections:

- The **definition is hidden** from users without ownership (visible in `SHOW VIEWS` / `GET_DDL` only to the owner role).
- The optimizer **will not push down user predicates** or reorder operations in ways that could leak data through error messages or timing, which closes side-channel information leaks.
- They are **required for sharing** (views must be secure) and recommended for row/column-level security patterns.

Trade-off: because of the restricted optimization, secure views can be **slower** than standard views, so use them where security demands it. For general access control today, masking and row access policies on tables are often preferred, with secure views for sharing.

Q71 How do replication groups and failover groups support business continuity?

- **Replication groups:** define a set of objects (databases, shares, users, roles, warehouses, integrations, parameters) replicated from a source account to one or more target accounts in the same organization, on a schedule. Targets are **read-only secondary** copies, useful for data distribution and migration across regions/clouds.
- **Failover groups:** a replication group that also supports **failover**, so the target can be promoted to become the new primary (Business Critical+) for DR. Works with **Client Redirect** (a connection object with a stable URL that switches to the new primary) so applications reconnect without config changes.

Plan RPO by replication schedule, and RTO by promotion and redirect steps. Test failover regularly, replicate dependencies (roles, integrations), and account for replication, storage and data-transfer cost.

Q72 How do ACCOUNT_USAGE, Access History and Trust Center support auditing?

- `SNOWFLAKE.ACCOUNT_USAGE`**:** shared views with up to **365 days** of history (with some latency, 45 minutes to 3 hours): `QUERY_HISTORY`, `LOGIN_HISTORY`, `ACCESS_HISTORY`, `GRANTS_TO_ROLES/USERS`, `METERING_HISTORY`, `TABLE_STORAGE_METRICS`. `INFORMATION_SCHEMA` offers real-time but shorter history.
- **Access History (Enterprise+):** records which user queried which objects and **columns**, including base objects behind views, and write operations, enabling data lineage, sensitive-data access audits, and detecting unused tables.
- **Trust Center:** continuously evaluates the account against security best practices (CIS benchmark, threat-intelligence scanners), flags weak authentication, over-privileged roles and risky users, and sends notifications.

Combine them for audit reporting, compliance evidence and anomaly detection.

Snowflake AI / Cortex & ML

Q73 What is Snowflake Cortex, and which capabilities does it include?

Snowflake Cortex is Snowflake's fully managed AI layer, which brings LLMs, search and ML to your data **inside the Snowflake security and governance boundary**, with no infrastructure to manage.

Main capabilities:

- **LLM and AISQL functions:** `AI_COMPLETE`, `AI_CLASSIFY`, `AI_FILTER`, `AI_AGG`, `AI_EXTRACT`, `AI_SENTIMENT`, `AI_TRANSLATE`, `AI_EMBED` and others, callable from SQL.
- **Cortex Search:** hybrid vector and keyword search service for RAG and search apps.
- **Cortex Analyst:** natural-language to SQL over structured data using semantic models/views.
- **Cortex Agents** and **Snowflake Intelligence:** orchestration and a business-user chat experience.
- **Document processing:** `AI_PARSE_DOCUMENT`, Document AI.
- **Cortex Fine-tuning** and **Cortex Guard** for customization and safety.
- **Cortex ML functions** for forecasting, anomaly detection and classification.

Models include hosted open and partner models, and function names and availability evolve, so check current documentation.

Q74 Explain Cortex LLM functions and AISQL functions (COMPLETE, SUMMARIZE, TRANSLATE, SENTIMENT, classify/filter/aggregate/extract).

Cortex LLM functions run models on data directly from SQL, billed per token. AISQL extends this with **AI operators** that work on rows, columns, and multimodal data (text, images, documents).

- `AI_COMPLETE` (formerly `COMPLETE`): general prompt-to-response with a chosen model, supports parameters and structured output.
- `AI_SUMMARIZE_AGG` / `SUMMARIZE`: summarize text or aggregate summaries across rows.
- `AI_TRANSLATE` / `TRANSLATE`: language translation.
- `AI_SENTIMENT` / `SENTIMENT`: sentiment score or aspect-based sentiment.
- `AI_CLASSIFY`: classify text or images into your labels.
- `AI_FILTER`: returns a boolean from a natural-language predicate, so you can use it in `WHERE` or joins.
- `AI_AGG`: aggregates insights across many rows with a prompt, avoiding context limits.
- `AI_EXTRACT`: extract fields or answers from text and documents.

Example: `SELECT AI_CLASSIFY(review, ['bug','praise','feature request']) FROM reviews;`

Controls: batch, select cheaper models for simple tasks, and watch tokens.

Q75 How does Cortex Search work (hybrid vector and keyword search), and how does it support RAG?

Cortex Search is a managed search service over text columns of a table or query. You create it with `CREATE CORTEX SEARCH SERVICE`, naming the search column, attributes (filters), a warehouse and a `TARGET_LAG`. Snowflake chunk-level indexes the text, builds **embeddings** and a **keyword index**, and keeps them refreshed as the source changes.

At query time it runs **hybrid retrieval**: vector (semantic) search plus keyword (lexical) search, merges them, and applies **semantic reranking**, returning the best matching records, with attribute filters for access scoping.

For **RAG**, an app or agent sends the user question to the service, takes the top chunks, and passes them as context to `AI_COMPLETE` to generate a grounded answer with citations. It is exposed through SQL, Python and REST APIs and is a tool for Cortex Agents.

Q76 What is Cortex Analyst, and how does text-to-SQL work on structured data?

Cortex Analyst is a managed service (REST API) that answers business questions in natural language over **structured data** by generating SQL, executing it in your warehouse and returning results with the SQL and an explanation.

How it works: the question plus a **semantic model or semantic view** (which defines tables, relationships, dimensions, measures, synonyms, metrics and verified queries) is sent to a multi-model pipeline. It resolves business terms to columns and logic, writes SQL, validates it, and can ask clarifying questions. Because it works from the semantic layer, not raw schemas, it reduces ambiguity and hallucination. Data and SQL execution stay inside Snowflake, and governance policies (RBAC, masking) apply to the generated query. Use it for self-service analytics and embed it in apps, Slack or agents.

Q77 What are semantic models and semantic views, and why do they improve text-to-SQL accuracy?

A **semantic model** (YAML file) or **semantic view** (a native schema-level Snowflake object, `CREATE SEMANTIC VIEW`) describes the business meaning of your data: logical tables and columns, **dimensions, facts, metrics**, relationships and join paths, synonyms, descriptions, filters and sample values, plus verified queries.

Why it improves accuracy:

- Maps messy physical names to business terms ("revenue", "active customer").
- Encodes **canonical metric definitions** and joins once, so every question computes them the same way.
- Gives the LLM context and constraints, which reduces wrong joins and invented columns.
- Provides a governed, reusable layer for Cortex Analyst, agents, and BI tools.

Semantic views being native objects means they use RBAC, can be shared, versioned and queried with SQL. Good semantic design (clear descriptions, synonyms, verified queries) is the biggest accuracy lever.

Q78 What are Cortex Agents, and how do they orchestrate tools across structured and unstructured data?

Cortex Agents are managed agentic AI services that **plan, call tools, and reflect** to answer multi-step questions. You configure an agent with an orchestration model, instructions, and a set of **tools**:

- **Cortex Analyst** for structured data (text-to-SQL).
- **Cortex Search** for unstructured data (documents, tickets, emails).
- **Custom tools** (UDFs, stored procedures, external functions, MCP servers) for actions or calculations.

The agent breaks a request into steps, picks tools (for example Analyst for sales numbers, then Search for the policy document explaining them), combines the results, and returns an answer with citations, streaming progress through the REST API. Everything runs under the caller's role, so governance is enforced. Agents are the engine behind Snowflake Intelligence, and the basis for chatbots and workflow automation.

Q79 What is Snowflake Intelligence, and who is it for?

Snowflake Intelligence is a **no-code, conversational interface** where business users ask questions in natural language about their enterprise data and get answers, charts and insights, built on **Cortex Agents** (Analyst plus Search, plus custom tools).

It is for **business users, analysts and executives** who want self-service insights without writing SQL, while data teams build and govern the underlying agents, semantic views and search services. Because it runs inside Snowflake, answers respect each user's roles, masking and row policies, and it can span structured tables and unstructured documents, and with connectors to other apps. Admins control which agents are available to whom.

Q80 Design an end-to-end RAG pipeline in Snowflake (parsing, chunking, embeddings, retrieval, generation).

1. **Ingest:** land documents (PDF, DOCX, HTML) in a stage with a **directory table**, or ingest via Openflow connectors (SharePoint, Google Drive).
2. **Parse:** `AI_PARSE_DOCUMENT` (OCR or LAYOUT mode) extracts text, tables and structure.
3. **Chunk:** split text with `SNOWFLAKE.CORTEX.SPLIT_TEXT_RECURSIVE_CHARACTER` (about 500-1,500 characters with overlap), keeping metadata (source, page, section, ACL tags).
4. **Embed and index:** create a **Cortex Search service** over the chunk table (it embeds and indexes automatically) or store `VECTOR` embeddings made with `AI_EMBED` and query with similarity functions.
5. **Retrieve:** query the service (hybrid search) with the question and metadata filters (user entitlements).
6. **Generate:** pass top chunks and the question in a prompt to `AI_COMPLETE`, instructing it to answer only from the context and cite sources.
7. **Operate:** refresh via streams/dynamic tables, evaluate quality, monitor cost, and expose via Streamlit or an agent.

Q81 Explain the VECTOR data type, embedding functions and similarity functions (cosine, L2, inner product).

`VECTOR(FLOAT, n)` (or INT) is a fixed-dimension array type for embeddings (up to 4,096 dimensions), stored in tables and usable in SQL.

Embedding functions: `AI_EMBED` (and `EMBED_TEXT_768`, `EMBED_TEXT_1024`) convert text (and images in newer models) into vectors with a chosen embedding model.

Similarity/distance functions:

- `VECTOR_COSINE_SIMILARITY(a, b)`: angle-based similarity, from -1 to 1, the usual choice for text embeddings.
- `VECTOR_L2_DISTANCE(a, b)`: Euclidean distance, smaller is closer.
- `VECTOR_INNER_PRODUCT(a, b)`: dot product, equal to cosine for normalized vectors.
- `VECTOR_L1_DISTANCE` also exists.

```
SELECT id FROM docs ORDER BY VECTOR_COSINE_SIMILARITY(emb, :q) DESC LIMIT 5;
```

Use the same model for query and document embeddings, and for scale prefer Cortex Search, which manages indexing.

Q82 How do you process unstructured documents (Document AI, PARSE_DOCUMENT, AI_EXTRACT)?

- `AI_PARSE_DOCUMENT` (previously `PARSE_DOCUMENT`): reads files on a stage and returns text and structure. **OCR mode** for plain text extraction, **LAYOUT mode** preserving tables, headings and reading order (Markdown). Best first step for RAG and analytics.
- `AI_EXTRACT`**:** extracts specific fields or answers from documents and text using a schema or questions, returning JSON. Good for invoices, contracts and forms with a flexible layout and no training.
- **Document AI:** earlier model-based approach (Arctic-TILT) in which you build a model, define values to extract, review and optionally train on your documents, then run it with `<model>!PREDICT` in pipelines. Suited to high-volume, consistent document types needing review workflows.

Pattern: stage plus directory table, a stream/task or dynamic table to process new files, store structured outputs in tables with confidence and source, and add human review for critical fields.

Q83 When and how do you fine-tune a model with Cortex Fine-tuning?

Fine-tune when prompting and RAG cannot give consistent **style, format or domain-specific behavior**, or when a smaller, cheaper fine-tuned model can replace a bigger one for a narrow task (classification, extraction, tone), with lower latency and cost.

How: prepare a table of prompt and completion pairs (a few hundred to thousands of high-quality examples), split into training and validation, and run `SNOWFLAKE.CORTEX.FINETUNE('CREATE', 'my_model', 'base_model', 'train_query', 'validation_query')`. Monitor with `FINETUNE('DESCRIBE', job_id)`. The result is a model object in your account (private to you) that you call with `AI_COMPLETE('my_model', prompt)`. Training is billed per token trained, and base models are limited to supported ones.

Try prompt engineering and RAG first, because fine-tuning does not add reliable new knowledge and needs good data and evaluation.

Q84 How do Cortex Guard and Snowflake's data-governance boundary address AI safety and privacy? What about cross-region inference?

- **Cortex Guard:** enable the `guardrails` option on `AI_COMPLETE` so a safety model filters unsafe inputs and outputs (violent, hateful, self-harm, sexual content, and similar), returning a blocked response instead.
- **Governance boundary:** models are hosted within Snowflake's infrastructure, so your data is not sent to third parties for training or retained by model providers, and prompts and outputs are subject to your RBAC, masking, row access policies, auditing and regional compliance. Access to functions is controlled by the `SNOWFLAKE.CORTEX_USER` database role.
- **Cross-region inference:** if a model is not available in your region, enable `CORTEX_ENABLED_CROSS_REGION` so requests can be processed in another region (within configured geographies). Data is encrypted in transit and not stored, but it leaves your region, so assess residency requirements (GDPR, sovereignty) and keep it off if not allowed.

Q85 How do you monitor and control Cortex AI cost (token-based billing, usage history views, batching, model choice)?

Billing: AI functions are billed by **tokens** (input and output) at per-model rates in credits per million tokens. Cortex Search, Analyst and Agents have their own usage meters (for example search serving plus indexing).

Monitor: `SNOWFLAKE.ACCOUNT_USAGE` views such as `CORTEX_FUNCTIONS_USAGE_HISTORY`, `CORTEX_FUNCTIONS_QUERY_USAGE_HISTORY`, `CORTEX_SEARCH_SERVING_USAGE_HISTORY`, `CORTEX_ANALYST_USAGE_HISTORY` (and `METERING_DAILY_HISTORY` for service types), plus budgets and tags for alerts.

Control:

- Pick the **smallest model** that meets quality, since costs differ widely.
- Reduce tokens: shorter prompts, trimmed context (fewer chunks), `max_tokens`.
- Run on **filtered rows** (avoid calling the LLM for all rows), cache results in tables, and batch.
- Restrict model access with `CORTEX_MODELS_ALLOWLIST` and roles, and use budgets with alerts.

Q86 What is Snowflake ML, and how do distributed training and the container runtime work?

Snowflake ML is the integrated set of capabilities for building, deploying and managing ML in Snowflake: Snowpark ML modeling APIs, **Feature Store**, **Model Registry**, **Notebooks**, ML Observability, plus Cortex ML functions.

The **Container Runtime** runs notebooks and ML jobs on **Snowpark Container Services** compute pools (CPU or GPU), with preinstalled ML libraries (PyTorch, XGBoost, LightGBM, scikit-learn) and the ability to install from pip. It supports **distributed training and processing** across multiple nodes and GPUs through Snowflake ML's distributed APIs (for example distributed XGBoost/LightGBM/PyTorch estimators and distributed data loading), so you can train on large data without moving it out of Snowflake.

ML Jobs let you push scripts from any IDE to run as managed jobs. Workflow: develop in a notebook, train on a compute pool, log to the Model Registry, deploy for inference.

Q87 Explain the Model Registry and model monitoring/observability.

The **Model Registry** is a schema-level store for ML models (scikit-learn, XGBoost, PyTorch, Hugging Face, custom) with **versions**, metadata, metrics, a default version and aliases, governed by RBAC. You log with `reg.log_model(...)`, and run inference via Python (`mv.run(...)`) or SQL (`MODEL!PREDICT`), in a warehouse or as a service on SPCS (including GPU) with a REST endpoint.

Model monitoring (ML Observability) tracks deployed models by comparing baseline and production data: **drift** (feature and prediction distribution), **performance** (accuracy, precision/recall, RMSE when ground truth arrives), volume, and data quality. It stores results in tables, supports alerts, and shows dashboards in Snowsight. This gives lineage from data to model to predictions, and helps decide when to retrain.

Q88 What is the Snowflake Feature Store, and how does it prevent training/serving skew?

The Feature Store is a managed layer for creating, storing and serving ML features as Snowflake objects. You define **entities** (customer, product with join keys) and **feature views** (feature logic as a SQL or DataFrame query), which Snowflake materializes as **dynamic tables** refreshed on a schedule, and can serve online through an online feature store for low-latency lookups.

Preventing skew: the same feature definition is used for **offline training and online/batch inference**, so the logic is defined once and not reimplemented. It also gives **point-in-time correct joins** (`generate_training_set` with timestamps) to avoid data leakage, plus versioning, discovery, lineage and governance, so features are reusable across teams and consistently computed.

Q89 When do you use Snowpark Container Services and GPU compute pools for custom AI workloads?

Snowpark Container Services (SPCS) runs **containerized applications** inside Snowflake on managed **compute pools**, with images in an image repository, and either long-running **services** or one-off **jobs**.

Use it when workloads exceed what warehouses or Cortex provide:

- **Custom model training and inference** needing **GPUs** (deep learning, fine-tuning open models, LLM serving with vLLM, computer vision).
- Specialized runtimes, libraries or languages that UDFs do not support.
- Hosting APIs, web apps, vector databases, notebooks or third-party software next to the data.

Choose the compute pool instance family (CPU, high-memory, GPU) by workload, set auto-suspend and min/max nodes to control cost, and use network rules and secrets for egress. Prefer Cortex functions for standard LLM tasks, and SPCS for custom models.

Q90 What are Cortex ML functions (forecasting, anomaly detection, classification), and when do you use them over custom models?

Cortex ML functions are SQL-callable, managed ML functions that you train and run without writing ML code:

- **Forecasting:** time-series models predicting future values (with series by group, exogenous variables, prediction intervals).
- **Anomaly detection:** flags unusual points in time series.
- **Classification:** trains a model on labeled table data to predict categories.
- **Top Insights:** identifies drivers of a metric change.

Usage: create a model object (`CREATE SNOWFLAKE.ML.FORECAST m(INPUT_DATA => ..., TIMESTAMP_COLNAME => ..., TARGET_COLNAME => ...)`), then call `m!FORECAST(...)`.

Use them for fast, standard use cases, analysts without ML skills, and prototypes. Use **custom models** (Snowflake ML) when you need custom architectures, specialized features, tuning, strict control of evaluation, or complex deep learning.

Q91 How do Snowflake Notebooks support data science workflows?

Snowflake Notebooks are interactive, cell-based notebooks inside Snowsight (Python, SQL, Markdown) that run directly on Snowflake data, with no data movement or separate environment.

They run on a **warehouse runtime** (small data, SQL/Snowpark) or the **container runtime** (CPU/GPU compute pools for ML and large-scale Python), support Streamlit visualizations, packages from the Anaconda channel or pip, **Git integration**, scheduling as tasks, and sharing with RBAC. Typical workflow: explore with SQL and Snowpark, engineer features, train models (Snowflake ML), log them to the Model Registry, and schedule pipelines, or call Cortex functions. They shorten the path from analysis to production and keep all of it governed.

Q92 How do Streamlit in Snowflake and Native Apps help you ship AI applications?

Streamlit in Snowflake hosts Python Streamlit apps inside Snowflake, running as the viewing user's role (or owner's), against your data. You build chat UIs, dashboards, and tools quickly, calling Cortex Search, Analyst, `AI_COMPLETE` and the Model Registry, with no separate hosting, auth or data export.

Native Apps package data, code (UDFs, procedures, Streamlit, SPCS containers) and AI features into an app distributed through **listings and the Marketplace**. Consumers install it in their account, so their data stays put, while providers manage versions and can monetize.

Together they let internal teams ship governed AI tools quickly, and vendors distribute AI products to customers securely without data leaving the customer's account.

Q93 How do you govern AI usage (access to Cortex models, allow-listed models, masking PII before LLM calls)?

- **Access control:** Cortex functions need the `SNOWFLAKE.CORTEX_USER` database role (granted to PUBLIC by default, so revoke it and grant to selected roles), and `CORTEX_EMBED_USER` for embeddings. Control agents, search services and semantic views with normal RBAC.
- **Allow-listed models:** set the `CORTEX_MODELS_ALLOWLIST` parameter (account level) and role-based model access to restrict which models can be called.
- **PII protection:** apply masking policies and row access policies to source columns (applied automatically in queries that feed prompts), tag-based masking with classification, and de-identify or redact before prompts when needed, for example with `AI_COMPLETE` or `AI_REDACT`-style functions.
- **Safety and compliance:** Cortex Guard, control cross-region inference, audit usage with `ACCOUNT_USAGE`, budgets and Access History, and document lineage and approvals.

Q94 How do you evaluate LLM output quality and reduce hallucinations (evals, grounding, AI observability)?

Reduce hallucinations:

- **Ground** answers with retrieval (Cortex Search) or semantic views (Analyst), and instruct the model to answer only from context and say "I don't know" otherwise.
- Provide structured output (JSON schema), lower temperature, and cite sources.
- Use verified queries and good semantic descriptions for text-to-SQL.

Evaluate:

- Build a **golden dataset** of questions with expected answers or SQL.
- Measure **groundedness, answer relevance, context relevance** and correctness using an **LLM-as-judge** (for example with `AI_COMPLETE`) plus human review, and track retrieval metrics (recall, precision).
- **AI observability** (TruLens-based tracing and evaluations in Snowflake) logs prompts, retrieved contexts, outputs, latency and cost, so you compare versions and detect regressions.

Run evals in CI before releases and sample production traffic with user feedback.

Scenario-Based

Q95 Design a cost-effective warehouse strategy for ETL, BI and data science workloads.

Principles: isolate by workload, right-size by measurement, keep warehouses off when idle, and track spend per team.

- **ETL/ELT:** one or more medium/large warehouses, single-cluster, auto-suspend 60 s, scheduled jobs consolidated (fewer start-ups), tested on Gen2 for DML-heavy work. Staging as transient tables.
- **BI:** multi-cluster auto-scale (min 1, max by peak concurrency), Standard policy, auto-suspend 5-10 min for cache, rely on the **result cache** and materialized/dynamic aggregates, Query Acceleration for outliers.
- **Data science:** separate warehouse (Snowpark-optimized for training), strict timeouts, or container runtime compute pools with auto-suspend.
- **Ad hoc:** small, with `STATEMENT_TIMEOUT` and resource monitors.

Governance: resource monitors, budgets, tags/`QUERY_TAG` for chargeback, weekly review of idle time, queuing, spilling and credits per query, plus roles that restrict which warehouse each team can use.

Q96 How would you migrate a Teradata, Redshift or on-prem warehouse to Snowflake (assessment, code conversion, validation, cutover)?

1. **Assess:** inventory databases, tables, volumes, ETL jobs, reports, dependencies, users and SLAs. Use tools like SnowConvert AI to analyze complexity and sort workloads into lift-and-shift vs redesign.
2. **Design:** account/role/warehouse model, security, naming, data types, and the target architecture (medallion layers).
3. **Convert code:** DDL, views, stored procedures and ETL SQL with **SnowConvert**, then manual fixes for unsupported features (e.g., Teradata macros, QUALIFY nuances, Redshift distribution keys are dropped).
4. **Move data:** export to cloud storage (Parquet), load with `COPY INTO`, use CDC for ongoing changes, and repoint ETL/BI tools.
5. **Validate:** row counts, checksums/hashes, aggregates, sample comparisons, query result parity, performance benchmarks.
6. **Cutover:** parallel runs, phased migration by domain, rollback plan, user training, then decommission and optimize costs.

Q97 A query went from 10 seconds to 5 minutes. How do you investigate?

1. **Confirm and compare:** find both runs in `QUERY_HISTORY`. Compare elapsed time breakdown, bytes scanned, partitions scanned, rows produced, and warehouse used.
2. **Check what changed:** data volume growth, new data skew, table clustering degradation (`SYSTEM$CLUSTERING_INFORMATION`), schema/view/SQL change, a changed warehouse size or auto-suspend (cold cache), a different role/policy adding masking or row policies.
3. **Look at Query Profile:** pruning drop, spilling, exploding join, a plan change, or queued time (`QUEUED_OVERLOAD_TIME`) from concurrent load.
4. **Check the warehouse:** concurrency, queuing, resized or shared with a heavy job (`WAREHOUSE_LOAD_HISTORY`).

5. **Fix:** rewrite or add filters, recluster or add Search Optimization, resize or isolate, update statistics by reloading data order, or use QAS.
6. **Prevent:** alerts on query duration regressions.

Q98 How would you implement GDPR deletion and PII protection in Snowflake?

Know your data: classify and tag PII (`SYSTEM$CLASSIFY`, auto-classification, tags), keep a data map and use Access History for lineage.

Protect: masking policies (tag-based), row access policies, RBAC with least privilege, encryption, network policies and secure views/shares, pseudonymization or hashing where possible.

Right to erasure: locate the subject across tables via a key (`subject_id`), then `DELETE`/`MERGE` in all raw, silver, gold and derived tables, including ML features, search indexes, embeddings, stages and shares, and consider clones, replicas and exports.

Retention caveat: deleted data remains recoverable during **Time Travel and Fail-safe**, so define policy (short retention or transient tables for PII, document that data expires after the retention window, 7-day Fail-safe). Alternatively use **crypto-shredding** by encrypting per subject with application keys and deleting the key.

Also log deletion requests, audit, and test erasure end to end.

Q99 Design an enterprise chatbot over both structured tables and unstructured documents using Cortex.

Architecture:

1. **Structured:** model gold tables as a **semantic view** (metrics, dimensions, synonyms, verified queries) and expose it to **Cortex Analyst**.
2. **Unstructured:** store documents in a stage with a directory table, parse with `AI_PARSE_DOCUMENT`, chunk, and create a **Cortex Search service** (with ACL/department attributes for filtering). Keep it fresh with streams/dynamic tables.
3. **Orchestrate:** create a **Cortex Agent** with Analyst and Search as tools, plus custom tools (stored procedures to open tickets), instructions, and response style.
4. **Surface:** Snowflake Intelligence for business users, or a Streamlit/React app and Slack/Teams via the Agents REST API.
5. **Govern:** run as the user's role so masking and row policies apply, restrict models, enable Cortex Guard, log conversations.
6. **Operate:** evaluate with a golden question set, monitor cost and latency with budgets, collect user feedback, and iterate on semantic descriptions and chunking.

Q100 Design a near-real-time architecture covering ingestion, transformation and ML scoring in Snowflake.

Ingest: events from Kafka, apps or IoT enter via the **Snowpipe Streaming** (or Kafka connector in streaming mode, Openflow for CDC) into bronze tables with seconds of latency.

Transform: dynamic tables with short target lag (for example 1 minute) clean, dedupe, join and aggregate into silver/gold, or streams plus serverless tasks where procedural logic is needed. Use hybrid tables or the online Feature Store for low-latency lookups.

Score: features come from the **Feature Store** (feature views as dynamic tables), and the **Model Registry** model scores new rows either in SQL (`MODEL!PREDICT`) in a task/dynamic table or through an SPCS service for real-time APIs (GPU if needed). Cortex functions can add LLM enrichment (classification, sentiment).

Serve and act: write predictions to a table, drive dashboards, alerts and Streamlit apps, and monitor with ML Observability, freshness alerts and budgets.

Operate: dedicated warehouses, resource monitors, and replication for DR.